

Gap Remediation Plan

Project: EMS Readiness Optimization for Manhattan
Date: March 12, 2026
Reference: docs/project_alignment_verification.md

Overview

The project alignment verification identified **no critical gaps** and **three minor gaps** between the project outline and deliverables. All three gaps have been **fully resolved** in v1.1.0.

Gap Summary

ID	Gap	Priority	Effort	Status
G1	CBD robustness comparison not in DES experiments	⚠️ Recommended	Medium	✅ RESOLVED
G2	Queue metrics underreported in technical report	⚠️ Nice-to-have	Low	✅ RESOLVED
G3	Seasonal analysis not deeply covered	⚠️ Nice-to-have	Very Low	✅ RESOLVED

G1: CBD Robustness Comparison in DES

Problem

The outline states: “Manhattan analysis with CBD robustness comparison” (§3) and “Compare Manhattan-wide results with CBD-focused robustness results” (§11). While CBD data is analyzed in EDA and the spatial analysis, the four production DES experiments run only Manhattan-wide.

Recommended Fix: Post-Hoc CBD Filtering (Option B)

Rather than re-running simulations, filter the existing incident-level logs from production experiments to include only incidents in CBD precincts, then recompute metrics.

Implementation Steps

1. **Identify CBD precincts** — Use `data/raw/cbd_boundary.geojson` and `data/processed/precincts_manhattan.geojson` to identify which precincts overlap the CBD (approximately precincts 1, 5, 6, 7, 10, 13, 14, 17, 18, 19, 20).
2. **Create CBD analysis script** — `scripts/analyze_cbd_robustness.py` :
 - Load incident-level logs from production experiments
 - Filter incidents where `precinct` is in the CBD precinct set
 - Recompute: mean RT, P90 RT, 8-min coverage, utilization for CBD-only incidents
 - Compare P0 vs P1 vs P2 performance in CBD subset
 - Generate comparison table and figure
3. **Add CBD section to technical report** — Add a subsection in Results (§5) titled “CBD Robustness Check”:
 - Table comparing Manhattan-wide vs CBD metrics for each policy
 - Brief interpretation (e.g., “P2’s advantage is even more pronounced in the CBD due to higher demand concentration”)
4. **Update experimental design doc** — Add note that CBD analysis was performed as post-hoc filtering.

Estimated Effort: 2-3 hours

Prerequisites

- Incident-level logs must include precinct field (✓ they do)
- CBD precinct mapping exists (✓ `cbd_boundary.geojson` available)

✓ Resolution (v1.1.0)

Approach chosen: Dedicated CBD DES experiment (Option A — exceeding the recommended Option B).

Implementation:

- Identified 10 CBD precincts via spatial intersection ($\geq 30\%$ overlap with MTA Congestion Relief Zone): precincts 1, 5, 6, 7, 9, 10, 13, 14, 17, 18
- Created `docs/cbd_definition.md` documenting CBD precinct selection methodology
- Created `configs/cbd_scenario.yaml` with CBD-specific scenario parameters
- Built `scripts/run_cbd_experiment.py` — full factorial CBD experiment:
- 11 scenarios $\times$ 3 policies $\times$ 30 replications = **330 dedicated CBD simulation runs**
- Scenarios: baseline, high-demand CBD, low-demand non-CBD, fleet variations (K=15,20,25,30), service time variations
- Created `notebooks/09_cbd_analysis.ipynb` for interactive CBD analysis
- Created `docs/cbd_robustness_analysis.md` with comprehensive findings
- Added §5.7 “CBD Robustness Analysis” to `docs/technical_report.md`

Key findings: P2 dominates across all CBD scenarios. CBD response times are 2.5–2.9 min for all policies due to firehouse concentration. P0 degrades to 12.81 min in non-CBD areas. P2’s advantage is most pronounced in mixed CBD/non-CBD scenarios.

Deliverables: 3 figures (`cbd_response_comparison.png` , `cbd_scenario_comparison.png` , `cbd_heatmap.png`), 2 tables (`cbd_summary_all.csv` , `cbd_comparison.csv`), 1 notebook, 2 docs.

G2: Queue Metrics in Technical Report

Problem

Queue length (mean, max) and queue fraction are tracked in every simulation run but not prominently reported in the technical report or publication tables.

Recommended Fix

Add queue metrics to the results section.

Implementation Steps

1. **Extract queue data** from `results/simulation/production/exp1_policy_comparison.csv` :
 - Columns: `mean_queue_length` , `max_queue_length` , `queue_fraction` , `incidents_queued`

2. **Add to technical report** — In §5 (Results), add a paragraph:

“Queue behavior was minimal across all policies under the K=20 baseline scenario. Under P2, mean queue length was [X] with [Y]% of incidents experiencing any queuing, compared to [A] and [B]% under P0.”

3. **Optionally add table** — A small table in the appendix showing queue metrics by policy.

Estimated Effort: 30 minutes

✓ Resolution (v1.1.0)

Implementation:

- Created `scripts/analyze_queue_metrics.py` — comprehensive queue analysis across all 1,770 simulation runs
- Analyzed queue metrics: mean queue length, max queue length, queue fraction, incidents queued
- Created `docs/queue_analysis.md` with full findings
- Added §5.8 “Queueing Performance Analysis” to `docs/technical_report.md`

Key findings: Queue metrics are **zero across all 1,770 runs**. System utilization is 10–15%, far below queueing thresholds. This is a legitimate and important finding — the fleet is sufficiently sized that no incidents experience queueing delays under any tested scenario. This validates the system design and demonstrates that response time differences between policies are driven by spatial allocation, not capacity constraints.

Deliverables: 4 figures (`queue_comparison_by_policy.png` , `queue_vs_fleet_size.png` , `queue_vs_demand.png` , `queue_heatmap.png`), 2 tables (`queue_statistics.csv` , `queue_anova.csv`), 1 doc.

G3: Seasonal Analysis

Problem

The outline mentions analysis by “season” in §6. The project covers hourly and DOW patterns thoroughly but does not explicitly discuss seasonal/monthly trends.

Recommended Fix

Add a brief seasonal note to the demand model documentation.

Implementation Steps

1. **Check existing EDA** — `notebooks/02_eda_spatiotemporal.ipynb` likely includes monthly/annual trends. If so, reference it.

2. **Add sentence to demand model spec** — In `demand_model_spec.md`, add:

“Monthly analysis reveals relatively stable crash rates across seasons, with a slight increase during summer months (June–August) and a modest decrease in winter (December–February). These seasonal effects are small relative to hourly and DOW variation and are subsumed by the NHPP model’s average rate.”

3. **If data supports it**, add a brief monthly bar chart to the EDA notebook.

Estimated Effort: 15–30 minutes

✓ Resolution (v1.1.0)

Implementation:

- Created `scripts/analyze_seasonal_patterns.py` — monthly/seasonal pattern analysis from 2.24M crash records
- Computed monthly demand factors, seasonal decomposition, and statistical tests
- Created `docs/demand_model_spec.md` §8 “Seasonal Patterns” with monthly factors and chi-square results
- Added §5.9 “Seasonal Variation Analysis” to `docs/technical_report.md`

Key findings: Monthly coefficient of variation is **9%** — moderate seasonal variation. Peak month is October (factor 1.103), trough is February (factor 0.822). Chi-square test rejects strict uniformity ($p < 0.001$), but the 9% CV indicates variation is small relative to hourly (CV ~60%) and DOW patterns. The NHPP model using annual averages remains appropriate, with seasonal factors documented for future refinement.

Deliverables: 3 figures (`seasonal_patterns.png` , `seasonal_decomposition.png` , `seasonal_heatmap.png`), 1 table (`seasonal_analysis.csv`), updated `demand_model_spec.md`.

Outcome

All three gaps are now **fully resolved**. The project achieves **100% alignment** with the project outline.

Gap	Resolution	New Runs	New Figures	New Tables
G1	CBD DES experiment	330	3	2
G2	Queue analysis	0	4	2
G3	Seasonal analysis	0	3	1
Total		330	10	5

Tagged as v1.1.0 — Gap Closure Release.